

Punto 2.7 - Configuración mediante Fluent API

Ejercicio guiado: recuperar y centralizar con Fluent API las reglas del modelo que en 2.6 se habían expresado parcialmente mediante Data Annotations, comprobando qué cambia en el modelo relacional y por qué Fluent API prevalece cuando ambas fuentes configuran la misma característica.

Contexto del proyecto: 2.7 parte íntegramente del estado 2.6. Las Data Annotations continúan presentes en las clases, pero OnModelCreating vuelve a expresar explícitamente propiedades, defaults y relaciones. Este punto no introduce todavía índices avanzados, restricciones CHECK ni filtros globales: esos conceptos se reservan para sus puntos específicos.

Objetivos de aprendizaje

- Configurar tablas y claves con ToTable y HasKey.
- Configurar propiedades con Property, IsRequired y HasMaxLength.
- Restaurar defaults de base de datos con HasDefaultValue y HasDefaultValueSql.
- Configurar precisión decimal con HasPrecision.
- Configurar relaciones 1:N y 1:1 con HasOne, WithMany, WithOne y HasForeignKey.
- Configurar DeleteBehavior de forma explícita.
- Mantener la clave compuesta y relaciones de OrdenAleacion.
- Comprender por qué Fluent API tiene prioridad sobre Data Annotations.
- Interpretar la migración real M2_2_7.
- Comprobar el modelo efectivo mediante context.Model.
- Trasladar una regla MaxLength desde annotation a Fluent sin cambiar el resultado final.

Paso 1: Abrir el estado autónomo de 2.7

```
cd M02/PROYECTO/2.7
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

El checkpoint contiene el estado completo del punto. Si reproduces la evolución manualmente, parte de 2.6.

Paso 2: Revisar la coexistencia con Data Annotations

Las clases conservan los atributos añadidos en 2.6, por ejemplo:

```
[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }

    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;

    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;
}
```

La Fluent API no obliga a eliminar estos atributos. Pueden coexistir. Si ambos mecanismos expresan la misma regla, el modelo final es coherente; si expresan valores diferentes, la configuración Fluent aplicada en OnModelCreating prevalece.

Paso 3: Configurar OrdenFabricacion con Fluent API

El bloque central del punto es:

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.ToTable("OrdenesFabricacion");
    entity.HasKey(o => o.Id);

    entity.Property(o => o.NumeroOrden)
        .IsRequired()
        .HasMaxLength(50);

    entity.Property(o => o.Cliente)
        .IsRequired()
        .HasMaxLength(200);

    entity.Property(o => o.FechaCreacion)
        .HasDefaultValueSql("GETDATE()");

    entity.Property(o => o.Estado)
        .IsRequired()
        .HasMaxLength(50)
        .HasDefaultValue("Pendiente");

    entity.Property(o => o.Observaciones)
        .HasMaxLength(500);
});
```

ToTable y HasKey hacen explícitos tabla y PK. Property selecciona una propiedad concreta. IsRequired y HasMaxLength describen nulabilidad y longitud. HasDefaultValueSql delega el valor por defecto a SQL Server; HasDefaultValue define un valor constante del esquema.

Paso 4: Comprender la diferencia entre inicializador C# y default SQL

En la clase puede existir:

```
public string Estado { get; set; } = "Pendiente";
```

pero esa asignación sólo actúa cuando .NET crea una instancia. Fluent API añade además:

```
entity.Property(o => o.Estado)
    .HasDefaultValue("Pendiente");
```

Este segundo valor forma parte del modelo relacional y de la migración. Lo mismo ocurre con:

```
entity.Property(o => o.FechaCreacion)
    .HasDefaultValueSql("GETDATE()");
```

Por eso 2.7 produce cambios de esquema aunque los objetos C# ya tengan inicializadores.

Paso 5: Restaurar precisión y defaults de PlanchaAcero

```
modelBuilder.Entity<PlanchaAcero>(entity =>
{
    entity.ToTable("PlanchasAcero");
    entity.HasKey(x => x.Id);

    entity.Property(x => x.Peso)
        .HasPrecision(18, 3);

    entity.Property(x => x.Activa)
        .HasDefaultValue(true);

    entity.HasOne(x => x.Orden)
        .WithMany(o => o.Planchas)
        .HasForeignKey(x => x.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
```

Aquí Fluent API complementa la annotation Precision y vuelve a introducir el DEFAULT SQL de Activa.

Paso 6: Configurar Aleacion y EstadoOrden

```
modelBuilder.Entity<Aleacion>(entity =>
{
    entity.ToTable("Aleaciones");
    entity.HasKey(a => a.Id);
    entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
    entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
    entity.Property(a => a.Descripcion).HasMaxLength(500);
});

modelBuilder.Entity<EstadoOrden>(entity =>
{
    entity.ToTable("EstadosOrden");
    entity.HasKey(e => e.Id);
    entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
    entity.Property(e => e.Descripcion).HasMaxLength(250);
    entity.Property(e => e.Activo).HasDefaultValue(true);
});
```

El default true de Activo vuelve a formar parte del esquema de SQL Server.

Paso 7: Mantener las relaciones uno-a-uno

DetalleOrden conserva:

```
entity.HasOne(d => d.Orden)
    .WithOne(o => o.Detalle)
    .HasForeignKey<DetalleOrden>(d => d.OrdenId)
    .OnDelete(DeleteBehavior.Cascade)
    .IsRequired();
```

CertificadoCalidad usa la misma estructura con su propia FK. Fluent API es especialmente útil aquí porque permite señalar de forma inequívoca el dependiente, la FK y el comportamiento de eliminación.

Paso 8: Configurar OrdenAleacion de forma explícita

```
modelBuilder.Entity<OrdenAleacion>(entity =>
{
    entity.ToTable("OrdenesAleaciones");
    entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });

    entity.Property(x => x.FechaAsignacion)
        .HasDefaultValueSql("GETDATE()");

    entity.Property(x => x.CantidadUtilizada)
        .HasPrecision(18, 3);

    entity.Property(x => x.EstadoRelacion)
        .IsRequired()
        .HasMaxLength(20)
        .HasDefaultValue("Activa");

    entity.HasOne(x => x.Orden)
        .WithMany(o => o.OrdenesAleaciones)
        .HasForeignKey(x => x.OrdenFabricacionId)
        .OnDelete(DeleteBehavior.Cascade);

    entity.HasOne(x => x.Aleacion)
        .WithMany(a => a.OrdenesAleaciones)
        .HasForeignKey(x => x.AleacionId)
        .OnDelete(DeleteBehavior.Restrict);
});
```

La configuración mantiene la PK compuesta de 2.5/2.6, restaura el default SQL de FechaAsignacion y el default de EstadoRelacion y conserva Cascade hacia Orden y Restrict hacia Aleacion.

Paso 9: Compilar antes de generar la migración

```
dotnet build AceriaData.sln --configuration Release
```

El build debe ser correcto antes de comparar el modelo con el snapshot de 2.6.

Paso 10: Generar M2_2_7

```
dotnet ef migrations add M2_2_7
```

El checkpoint ya contiene:

```
Migrations/20260927204806_M2_2_7.cs
```

No vuelvas a crear la misma migración dentro del estado final.

Paso 11: Revisar la migración real M2_2_7

La migración restaura exactamente los defaults que 2.6 había retirado:

- PlanchasAcero.Activa vuelve a tener default true.
- OrdenesFabricacion.FechaCreacion vuelve a tener default SQL GETDATE().
- OrdenesFabricacion.Estado vuelve a tener default "Pendiente".
- OrdenesAleaciones.FechaAsignacion vuelve a tener default SQL GETDATE().
- OrdenesAleaciones.EstadoRelacion vuelve a tener default "Activa".
- EstadosOrden.Activo vuelve a tener default true.

Por ejemplo:

```
migrationBuilder.AlterColumn<DateTime>(
    name: "FechaCreacion",
    table: "OrdenesFabricacion",
    type: "datetime2",
    nullable: false,
    defaultValueSql: "GETDATE()",
    oldClrType: typeof(DateTime),
    oldType: "datetime2");
```

La migración es la evidencia tangible de que Fluent API no es sólo una sintaxis alternativa: puede cambiar el metadata relacional que EF Core compara con el snapshot.

Paso 12: Aplicar y verificar el historial

```
dotnet ef database update
dotnet ef migrations list
```

La lista debe incluir M2_2_7 después de M2_2_6.

Paso 13: Ejecutar el estado y comprobar el default efectivo

```
dotnet run --project AceriaData.Console.csproj --configuration Release
```

La salida del checkpoint contiene:

```
2.7 OK | Fluent API | Estado default: Pendiente
```

La aplicación obtiene ese valor desde context.Model, no desde una constante escrita para la demostración.

Paso 14: Resolver el reto MaxLength de annotation a Fluent

Reto: trasladar la regla MaxLength de Cliente desde la annotation a Fluent API sin cambiar el modelo efectivo.

El checkpoint contiene el bloque RETO 2.7 - MAXLENGTH DE ANNOTATION A FLUENT. Para reproducirlo, comenta temporalmente:

```
[MaxLength(200)]
```

sobre Cliente. Mantén activa la regla Fluent:

```
entity.Property(o => o.Cliente)
    .IsRequired()
    .HasMaxLength(200);
```

y consulta el metadata:

```
var clienteMetadata = entity.FindProperty(nameof(OrdenFabricacion.Cliente))
    ?? throw new InvalidOperationException("No se encontró Cliente en el modelo.");

Console.WriteLine(
    $"Reto 2.7 MaxLength Cliente: {clienteMetadata.GetMaxLength()}");
```

Resultado esperado:

```
Reto 2.7 MaxLength Cliente: 200
```

Esto demuestra que la misma regla puede vivir en Fluent API sin alterar el modelo final.

Paso 15: Delimitar el alcance del punto

En 2.7 no se añaden:

- índices compuestos o únicos nuevos;
- restricciones CHECK;
- filtros globales HasQueryFilter;
- collation;
- owned types;
- table splitting;
- nuevas entidades.

Esos conceptos tienen sus propios puntos y se introducirán cuando corresponda. Aquí el objetivo es dominar la sintaxis y la prioridad de Fluent API sobre el modelo acumulado existente.

Errores comunes

Error	Causa	Corrección
Fluent contradice una annotation sin intención	Dos fuentes configuran valores distintos	Elegir conscientemente la regla final; Fluent prevalece
Se confunde inicializador C# con default SQL	Se observa el valor del objeto y no el esquema	Revisar la migración y el metadata del modelo
Se elimina PrimaryKey de OrdenAleacion sin mantener HasKey	Se rompe la PK compuesta	Conservar al menos una configuración correcta y coherente
Se cambia Cascade/Restrict al centralizar la configuración	Se altera el comportamiento de relaciones ya certificado	Mantener los DeleteBehavior heredados
Se añaden índices o CHECK	Se adelantan puntos posteriores	Reservarlos para sus apartados específicos
Se usa EnsureCreated()	Se evita el historial de Migrations	Mantener Database.Migrate() / database update

Analogía final

La Fluent API funciona como el plano central de configuración de la planta. Las etiquetas de las Data Annotations siguen pegadas a cada pieza, pero el plano central puede confirmar o sobrescribir una especificación y, además, describir relaciones completas entre componentes. La migración muestra qué decisiones del plano terminan afectando realmente a la instalación.

Resultado esperado

Al finalizar 2.7, AceriaData conserva las Data Annotations de 2.6 pero centraliza de nuevo propiedades, defaults y relaciones en Fluent API. M2_2_7 restaura los defaults SQL retirados en el punto anterior y la ejecución confirma que Estado tiene el default Pendiente. El reto demuestra que MaxLength de Cliente puede trasladarse de annotation a Fluent sin cambiar el resultado efectivo.

Conexión con el siguiente punto

2.8 partirá de este estado para trabajar específicamente con claves primarias, alternativas y compuestas, sin rehacer el proyecto.

| Código acumulativo completo del estado 2.7

El siguiente archivo es el `Program.cs` real del estado ejecutable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    [Required]
    [MaxLength(50)]
    public string Estado { get; set; } = "Pendiente";
    [MaxLength(500)]
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("PlanchasAcero")]
public class PlanchaAcero
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    [Precision(18, 3)]
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("Aleaciones")]
public class Aleacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;
    [Required]
    [MaxLength(20)]
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    [MaxLength(500)]
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }

```

```

    [Required]
    [MaxLength(50)]
    public string Nombre { get; set; } = string.Empty;
    [MaxLength(250)]
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    [MaxLength(500)]
    public string? Notas { get; set; }
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    [Required]
    [MaxLength(100)]
    public string OrganismoCertificador { get; set; } = string.Empty;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    [Precision(18, 3)]
    public decimal CantidadUtilizada { get; set; }
    [MaxLength(20)]
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>

```

```

{
    entity.ToTable("OrdenesFabricacion");
    entity.HasKey(o => o.Id);
    entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
    entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
    entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
    entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
    entity.Property(o => o.Observaciones).HasMaxLength(500);
});
modelBuilder.Entity<PlanchaAcero>(entity =>
{
    entity.ToTable("PlanchasAcero");
    entity.HasKey(x => x.Id);
    entity.Property(x => x.Peso).HasPrecision(18, 3);
    entity.Property(x => x.Activa).HasDefaultValue(true);
    entity.HasOne(x => x.Orden)
        .WithMany(o => o.Planchas)
        .HasForeignKey(x => x.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
modelBuilder.Entity<Aleacion>(entity =>
{
    entity.ToTable("Aleaciones");
    entity.HasKey(a => a.Id);
    entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
    entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
    entity.Property(a => a.Descripcion).HasMaxLength(500);
});
modelBuilder.Entity<EstadoOrden>(entity =>
{
    entity.ToTable("EstadosOrden");
    entity.HasKey(e => e.Id);
    entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
    entity.Property(e => e.Descripcion).HasMaxLength(250);
    entity.Property(e => e.Activo).HasDefaultValue(true);
});
modelBuilder.Entity<DetalleOrden>(entity =>
{
    entity.ToTable("DetallesOrden");
    entity.HasKey(d => d.Id);
    entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
    entity.Property(d => d.Notas).HasMaxLength(500);
    entity.HasOne(d => d.Orden)
        .WithOne(o => o.Detalle)
        .HasForeignKey<DetalleOrden>(d => d.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
modelBuilder.Entity<CertificadoCalidad>(entity =>
{
    entity.ToTable("CertificadosCalidad");
    entity.HasKey(c => c.Id);
    entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
    entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
    entity.HasOne(c => c.Orden)
        .WithOne(o => o.Certificado)
        .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
modelBuilder.Entity<OrdenAleacion>(entity =>
{
    entity.ToTable("OrdenesAleaciones");
    entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
    entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
    entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
    entity.Property(x =>
x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
    entity.HasOne(x => x.Orden)
        .WithMany(o => o.OrdenesAleaciones)
        .HasForeignKey(x => x.OrdenFabricacionId)

```

```

        .onDelete(DeleteBehavior.Cascade);
entity.HasOne(x => x.Aleacion)
    .WithMany(a => a.OrdenesAleaciones)
    .HasForeignKey(x => x.AleacionId)
    .onDelete(DeleteBehavior.Restrict);
    });
}
}

public interface IOordenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOordenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o =>
o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) =>
_context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOordenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOordenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>

```

```

        options
            .UseSqlServer(connectionString, sql =>
            {
                sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                sql.CommandTimeout(60);
            })
            .LogTo(global::System.Console.WriteLine, new[] {
"Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)
            .EnableDetailedErrors();

services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateScopes = true,
    ValidateOnBuild = true
});

using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();

var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    FechaEntrega = DateTime.Today.AddDays(14),
    Estado = "Pendiente",
    Observaciones = "Orden de validación M2",
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 371.250m,
            Activa = true,
        }
    },
    Detalle = new DetalleOrden
    {
        ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
        TemperaturaColada = 1550.5,
        Notas = "Colada principal"
    },
    Certificado = new CertificadoCalidad
    {
        NumeroCertificado = "CERT-0001",
        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    },
};

var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
    Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};
orden.OrdenesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacion,
    CantidadUtilizada = 1500.500m,
    EstadoRelacion = "Activa"
});
context.OrdenesFabricacion.Add(orden);
context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa",

```

```

    Activo = true });
    context.SaveChanges();

    var entity = context.Model.FindEntityType(typeof(OrdenFabricacion))!;
    global::System.Console.WriteLine($"2.7 OK | Fluent API | Estado default:
{entity.FindProperty(nameof(OrdenFabricacion.Estado))?.GetDefaultValue()}");
}
}

```

Explicación línea a línea del código acumulativo 2.7

Las referencias siguientes usan la numeración real de M02/PROYECTO/2.7/Program.cs. Se explican todas las líneas no vacías para que el documento sea trazable con el archivo que compila.

Línea 1: `using System.ComponentModel.DataAnnotations;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 2: `using System.ComponentModel.DataAnnotations.Schema;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 3: `using Microsoft.EntityFrameworkCore;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 4: `using Microsoft.Extensions.Configuration;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 5: `using Microsoft.Extensions.DependencyInjection;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 6: `using Microsoft.Extensions.Logging;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 8: `namespace AceriaData.ConsoleApp;` → declara el espacio de nombres del estado acumulativo.

Línea 10: `[Table("OrdenesFabricacion")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 11: `public class OrdenFabricacion` → declara la clase OrdenFabricacion que forma parte del estado acumulativo del ejercicio.

Línea 12: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 13: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 14: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 15: `[Required]` → marca la propiedad siguiente como requerida.

Línea 16: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 17: `public string NumeroOrden { get; set; } = string.Empty;` → declara la propiedad NumeroOrden de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 18: `[Required]` → marca la propiedad siguiente como requerida.

Línea 19: `[MaxLength(200)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 20: `public string Cliente { get; set; } = string.Empty;` → declara la propiedad Cliente de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 21: `public DateTime FechaCreacion { get; set; }` → declara la propiedad FechaCreacion de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.

Línea 22: `public DateTime? FechaEntrega { get; set; }` → declara la propiedad FechaEntrega de tipo DateTime? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 23: `[Required]` → marca la propiedad siguiente como requerida.

Línea 24: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 25: `public string Estado { get; set; } = "Pendiente";` → declara la propiedad Estado de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 26: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 27: `public string? Observaciones { get; set; }` → declara la propiedad Observaciones de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 28: `public List<PlanchaAcero> Planchas { get; set; } = new();` → declara la propiedad Planchas de tipo List que EF Core o la lógica de negocio utiliza en este modelo.

Línea 29: `public DetalleOrden? Detalle { get; set; }` → declara la propiedad Detalle de tipo DetalleOrden? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 30: `public CertificadoCalidad? Certificado { get; set; }` → declara la propiedad Certificado de tipo CertificadoCalidad? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 31: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad OrdenesAleaciones de tipo List que EF Core o la lógica de negocio utiliza en este modelo.

Línea 32: `}` → cierra el bloque de código actual.

Línea 34: `[Table("PlanchasAcero")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 35: `public class PlanchaAcero` → declara la clase PlanchaAcero que forma parte del estado acumulativo del ejercicio.

Línea 36: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 37: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 38: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 39: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 40: `public double Espesor { get; set; }` → declara la propiedad Espesor de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 41: `public double Ancho { get; set; }` → declara la propiedad Ancho de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 42: `public double Largo { get; set; }` → declara la propiedad Largo de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 43: `[Precision(18, 3)]` → configura precisión y escala para la propiedad numérica siguiente.

Línea 44: `public decimal Peso { get; set; }` → declara la propiedad Peso de tipo decimal que EF Core o la lógica de negocio utiliza en este modelo.

Línea 45: `public bool Activa { get; set; } = true;` → declara la propiedad Activa de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.

Línea 46: `[ForeignKey(nameof(OrdenId))]` → vincula explícitamente la navegación siguiente con su clave foránea.

Línea 47: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 48: `}` → cierra el bloque de código actual.

Línea 50: `[Table("Aleaciones")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

- Línea 51: `public class Aleacion` → declara la clase Aleacion que forma parte del estado acumulativo del ejercicio.
- Línea 52: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.
- Línea 53: `[Key]` → marca la propiedad siguiente como clave primaria simple.
- Línea 54: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
- Línea 55: `[Required]` → marca la propiedad siguiente como requerida.
- Línea 56: `[MaxLength(100)]` → limita la longitud máxima persistida de la propiedad siguiente.
- Línea 57: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
- Línea 58: `[Required]` → marca la propiedad siguiente como requerida.
- Línea 59: `[MaxLength(20)]` → limita la longitud máxima persistida de la propiedad siguiente.
- Línea 60: `public string Codigo { get; set; } = string.Empty;` → declara la propiedad Codigo de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
- Línea 61: `public double PorcentajeCarbono { get; set; }` → declara la propiedad PorcentajeCarbono de tipo double que EF Core o la lógica de negocio utiliza en este modelo.
- Línea 62: `public double PorcentajeManganeso { get; set; }` → declara la propiedad PorcentajeManganeso de tipo double que EF Core o la lógica de negocio utiliza en este modelo.
- Línea 63: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.
- Línea 64: `public string? Descripcion { get; set; }` → declara la propiedad Descripcion de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.
- Línea 65: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad OrdenesAleaciones de tipo List que EF Core o la lógica de negocio utiliza en este modelo.
- Línea 66: `}` → cierra el bloque de código actual.
- Línea 68: `[Table("EstadosOrden")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
- Línea 69: `public class EstadoOrden` → declara la clase EstadoOrden que forma parte del estado acumulativo del ejercicio.
- Línea 70: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.
- Línea 71: `[Key]` → marca la propiedad siguiente como clave primaria simple.
- Línea 72: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
- Línea 73: `[Required]` → marca la propiedad siguiente como requerida.
- Línea 74: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.
- Línea 75: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
- Línea 76: `[MaxLength(250)]` → limita la longitud máxima persistida de la propiedad siguiente.
- Línea 77: `public string Descripcion { get; set; } = string.Empty;` → declara la propiedad Descripcion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
- Línea 78: `public bool Activo { get; set; } = true;` → declara la propiedad Activo de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.
- Línea 79: `}` → cierra el bloque de código actual.

Línea 81: `[Table("DetallesOrden")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 82: `public class DetalleOrden` → declara la clase DetalleOrden que forma parte del estado acumulativo del ejercicio.

Línea 83: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 84: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 85: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 86: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 87: `[Required]` → marca la propiedad siguiente como requerida.

Línea 88: `[MaxLength(200)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 89: `public string ComposicionQuimica { get; set; } = string.Empty;` → declara la propiedad ComposicionQuimica de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 90: `public double TemperaturaColada { get; set; }` → declara la propiedad TemperaturaColada de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 91: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 92: `public string? Notas { get; set; }` → declara la propiedad Notas de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 93: `[ForeignKey(nameof(OrdenId))]` → vincula explícitamente la navegación siguiente con su clave foránea.

Línea 94: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 95: `}` → cierra el bloque de código actual.

Línea 97: `[Table("CertificadosCalidad")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 98: `public class CertificadoCalidad` → declara la clase CertificadoCalidad que forma parte del estado acumulativo del ejercicio.

Línea 99: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 100: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 101: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 102: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 103: `[Required]` → marca la propiedad siguiente como requerida.

Línea 104: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 105: `public string NumeroCertificado { get; set; } = string.Empty;` → declara la propiedad NumeroCertificado de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 106: `public DateTime FechaEmision { get; set; }` → declara la propiedad FechaEmision de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.

Línea 107: `[Required]` → marca la propiedad siguiente como requerida.

Línea 108: `[MaxLength(100)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 109: `public string OrganismoCertificador { get; set; } = string.Empty;` → declara la propiedad `OrganismoCertificador` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 110: `[ForeignKey(nameof(OrdenId))]` → vincula explícitamente la navegación siguiente con su clave foránea.

Línea 111: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad `Orden` de tipo `OrdenFabricacion` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 112: `}` → cierra el bloque de código actual.

Línea 114: `[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]` → declara una clave primaria compuesta con las propiedades indicadas.

Línea 115: `[Table("OrdenesAleaciones")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 116: `public class OrdenAleacion` → declara la clase `OrdenAleacion` que forma parte del estado acumulativo del ejercicio.

Línea 117: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 118: `public int OrdenFabricacionId { get; set; }` → declara la propiedad `OrdenFabricacionId` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 119: `public int AleacionId { get; set; }` → declara la propiedad `AleacionId` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 120: `public DateTime FechaAsignacion { get; set; } = DateTime.Now;` → declara la propiedad `FechaAsignacion` de tipo `DateTime` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 121: `[Precision(18, 3)]` → configura precisión y escala para la propiedad numérica siguiente.

Línea 122: `public decimal CantidadUtilizada { get; set; }` → declara la propiedad `CantidadUtilizada` de tipo `decimal` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 123: `[MaxLength(20)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 124: `public string EstadoRelacion { get; set; } = "Activa";` → declara la propiedad `EstadoRelacion` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 125: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad `Orden` de tipo `OrdenFabricacion` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 126: `public Aleacion Aleacion { get; set; } = null!;` → declara la propiedad `Aleacion` de tipo `Aleacion` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 127: `}` → cierra el bloque de código actual.

Línea 129: `public class AceriaDbContext : DbContext` → declara la clase `AceriaDbContext` que forma parte del estado acumulativo del ejercicio.

Línea 130: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 131: `public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }` → declara el constructor o método `AceriaDbContext` con los parámetros necesarios para este componente.

Línea 133: `public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 134: `public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 135: `public DbSet<Aleacion> Aleaciones => Set<Aleacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 136: `public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 137: `public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 138: `public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 139: `public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 142: `protected override void OnModelCreating(ModelBuilder modelBuilder)` → sobrescribe `OnModelCreating` para definir configuración explícita del modelo.

Línea 143: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 144: `base.OnModelCreating(modelBuilder);` → sobrescribe `OnModelCreating` para definir configuración explícita del modelo.

Línea 145: `modelBuilder.Entity<OrdenFabricacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 146: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 147: `entity.ToTable("OrdenesFabricacion");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 148: `entity.HasKey(o => o.Id);` → define la clave primaria de la entidad.

Línea 149: `entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 150: `entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 151: `entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 152: `entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");` → selecciona una propiedad escalar para continuar su configuración.

Línea 153: `entity.Property(o => o.Observaciones).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 154: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 155: `modelBuilder.Entity<PlanchaAcero>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 156: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 157: `entity.ToTable("PlanchasAcero");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 158: `entity.HasKey(x => x.Id);` → define la clave primaria de la entidad.

Línea 159: `entity.Property(x => x.Peso).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 160: `entity.Property(x => x.Activa).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 161: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 162: `.WithMany(o => o.Planchas)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 163: `.HasForeignKey(x => x.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 164: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 165: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 166: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 167: `modelBuilder.Entity<Aleacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 168: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 169: `entity.ToTable("Aleaciones");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 170: `entity.HasKey(a => a.Id);` → define la clave primaria de la entidad.

Línea 171: `entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 172: `entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);` → selecciona una propiedad escalar para continuar su configuración.

Línea 173: `entity.Property(a => a.Descripcion).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 174: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 175: `modelBuilder.Entity<EstadoOrden>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 176: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 177: `entity.ToTable("EstadosOrden");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 178: `entity.HasKey(e => e.Id);` → define la clave primaria de la entidad.

Línea 179: `entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 180: `entity.Property(e => e.Descripcion).HasMaxLength(250);` → selecciona una propiedad escalar para continuar su configuración.

Línea 181: `entity.Property(e => e.Activo).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 182: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 183: `modelBuilder.Entity<DetalleOrden>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 184: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 185: `entity.ToTable("DetallesOrden");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 186: `entity.HasKey(d => d.Id);` → define la clave primaria de la entidad.

Línea 187: `entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 188: `entity.Property(d => d.Notas).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 189: `entity.HasOne(d => d.Orden)` → inicia la configuración de una navegación de referencia.

Línea 190: `.WithOne(o => o.Detalle)` → establece el otro extremo de referencia de una relación uno-a-uno.

Línea 191: `.HasForeignKey<DetalleOrden>(d => d.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 192: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 193: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 194: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 196: `modelBuilder.Entity<CertificadoCalidad>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 197: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 198: `entity.ToTable("CertificadosCalidad");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 199: `entity.HasKey(c => c.Id);` → define la clave primaria de la entidad.

Línea 200: `entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 201: `entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 202: `entity.HasOne(c => c.Orden)` → inicia la configuración de una navegación de referencia.

Línea 203: `.WithOne(o => o.Certificado)` → establece el otro extremo de referencia de una relación uno-a-uno.

Línea 204: `.HasForeignKey<CertificadoCalidad>(c => c.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 205: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 206: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 207: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 208: `modelBuilder.Entity<OrdenAleacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 209: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 210: `entity.ToTable("OrdenesAleaciones");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 211: `entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });` → define la clave primaria de la entidad.

Línea 212: `entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 213: `entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 214: `entity.Property(x =>`

`x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");` → selecciona una propiedad escalar para continuar su configuración.

Línea 215: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 216: `.WithMany(o => o.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 217: `.HasForeignKey(x => x.OrdenFabricacionId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 218: `.OnDelete(DeleteBehavior.Cascade);` → define el comportamiento de eliminación de la relación.

Línea 219: `entity.HasOne(x => x.Aleacion)` → inicia la configuración de una navegación de referencia.

Línea 220: `.WithMany(a => a.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 221: `.HasForeignKey(x => x.AleacionId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 222: `.OnDelete(DeleteBehavior.Restrict);` → define el comportamiento de eliminación de la relación.

Línea 223: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 224: `}` → cierra el bloque de código actual.

Línea 225: `}` → cierra el bloque de código actual.

Línea 227: `public interface IOrdenRepositorio` → declara el contrato IOrdenRepositorio para desacoplar consumidores e implementación.

Línea 228: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 229: `List<OrdenFabricacion> ObtenerTodas();` → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.

Línea 230: `OrdenFabricacion? ObtenerPorId(int id);` → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.

Línea 231: `OrdenFabricacion? ObtenerPorNumero(string numeroOrden);` → declara en el contrato la búsqueda por el identificador de negocio NumeroOrden.

Línea 232: `void Agregar(OrdenFabricacion orden);` → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.

Línea 233: `void Eliminar(OrdenFabricacion orden);` → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.

Línea 234: `int Guardar();` → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.

Línea 235: `}` → cierra el bloque de código actual.

Línea 237: `public sealed class OrdenRepositorio : IOrdenRepositorio` → declara la clase OrdenRepositorio que forma parte del estado acumulativo del ejercicio.

Línea 238: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 239: `private readonly AceriaDbContext _context;` → declara el campo privado _context de tipo AceriaDbContext que conserva una dependencia de la clase.

Línea 240: `public OrdenRepositorio(AceriaDbContext context) => _context = context;` → declara el constructor o método OrdenRepositorio con los parámetros necesarios para este componente.

Línea 242: `public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → ordena el resultado de la consulta antes de materializarlo.

Línea 243: `public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);` → implementa la búsqueda por clave primaria mediante Find, aprovechando primero el ChangeTracker y después la base de datos.

Línea 244: `public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);` → recupera la primera coincidencia o null cuando no existe.

Línea 245: `public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el ChangeTracker para insertarla al guardar.

Línea 246: `public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);` → marca la entidad indicada para borrado al confirmar los cambios.

Línea 247: `public int Guardar() => _context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el DbContext.

Línea 248: `}` → cierra el bloque de código actual.

Línea 250: `public interface IServicioOrdenes` → declara el contrato IServicioOrdenes para desacoplar consumidores e implementación.

Línea 251: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 252: `string ObtenerResumen(int ordenId);` → declara el servicio que construye un resumen de una orden a partir de su identificador.

Línea 253: `}` → cierra el bloque de código actual.

Línea 255: `public sealed class ServicioOrdenes : IServicioOrdenes` → declara la clase ServicioOrdenes que forma parte del estado acumulativo del ejercicio.

Línea 256: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 257: `private readonly IOrdenRepositorio _repositorio;` → declara el campo privado _repositorio de tipo IOrdenRepositorio que conserva una dependencia de la clase.

Línea 258: `private readonly AceriaDbContext _context;` → declara el campo privado _context de tipo AceriaDbContext que conserva una dependencia de la clase.

Línea 259: `public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)` → declara el constructor o método ServicioOrdenes con los parámetros necesarios para este componente.

Línea 260: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 261: `_repositorio = repositorio;` → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.

Línea 262: `_context = context;` → guarda el DbContext inyectado para consultar los datos necesarios durante el servicio.

Línea 263: `}` → cierra el bloque de código actual.

Línea 265: `public string ObtenerResumen(int ordenId)` → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.

Línea 266: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 267: `var orden = _repositorio.ObtenerPorId(ordenId);` → consulta la orden mediante el repositorio y conserva el resultado anulable en una variable local.

Línea 268: `if (orden is null) return $"Orden {ordenId} no encontrada";` → evalúa la condición y ejecuta el bloque asociado sólo cuando se cumple.

Línea 269: `var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);` → cuenta las filas que cumplen la consulta construida.

Línea 270: `return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";` → devuelve el resultado calculado al código que invocó el método.

Línea 271: `}` → cierra el bloque de código actual.

Línea 272: `}` → cierra el bloque de código actual.

Línea 274: `public static class Program` → declara la clase Program que forma parte del estado acumulativo del ejercicio.

Línea 275: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 276: `public static void Main()` → declara el punto de entrada ejecutable de la aplicación de consola.

Línea 277: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 278: `var configuration = new ConfigurationBuilder()` → inicia la construcción de la configuración de la aplicación.

Línea 279: `.SetBasePath(AppContext.BaseDirectory)` → establece la ruta base usada para localizar los archivos de configuración.

Línea 280: `.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)` → incorpora appsettings.json a la configuración.

Línea 281: `.AddEnvironmentVariables()` → incorpora variables de entorno como fuente adicional de configuración.

Línea 282: `.Build();` → materializa la configuración o el objeto encadenado construido en las líneas anteriores.

Línea 284: `var connectionString = configuration.GetConnectionString("AceriaDB")` → obtiene la cadena de conexión AceriaDB desde la configuración.

Línea 285: `?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");` → interrumpe el arranque con un error explícito si la configuración no contiene ConnectionStrings:AceriaDB.

Línea 287: `var services = new ServiceCollection();` → crea la colección donde se registran las dependencias de la aplicación.

Línea 288: `services.AddDbContext<AceriaDbContext>(options =>` → registra el DbContext en el contenedor de inyección de dependencias.

Línea 289: `options` → continúa la configuración del DbContext sobre el parámetro options recibido por la lambda de AddDbContext.

Línea 290: `.UseSqlServer(connectionString, sql =>` → selecciona SQL Server como proveedor de EF Core.

Línea 291: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 292: `sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);` → habilita reintentos ante fallos transitorios del proveedor SQL Server.

Línea 293: `sql.CommandTimeout(60);` → establece el tiempo máximo permitido para los comandos SQL.

Línea 294: `})` → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.

Línea 295: `.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)` → configura la salida del logging de EF Core.

Línea 296: `.EnableDetailedErrors()`; → activa errores detallados para facilitar el diagnóstico del laboratorio.

Línea 298: `services.AddScoped<IOrdenRepositorio, OrdenRepositorio>()`; → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.

Línea 299: `services.AddScoped<IServicio0rdenes, Servicio0rdenes>()`; → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.

Línea 301: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → construye el ServiceProvider con validación de servicios y garantiza su liberación al salir del ámbito mediante using var.

Línea 302: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 303: `ValidateScopes = true,` → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.

Línea 304: `ValidateOnBuild = true` → obliga al contenedor a validar las resoluciones registradas al construir el ServiceProvider.

Línea 305: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 307: `using var scope = provider.CreateScope();` → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.

Línea 308: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → resuelve un servicio obligatorio desde el contenedor de dependencias.

Línea 309: `context.Database.EnsureDeleted();` → elimina la base del laboratorio para reproducir el escenario desde un estado limpio.

Línea 310: `context.Database.Migrate();` → aplica las migraciones pendientes y deja AceriaDB en el esquema del punto.

Línea 312: `var orden = new OrdenFabricacion` → crea la entidad OrdenFabricacion que concentra el grafo de datos del escenario acumulativo.

Línea 313: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 314: `NumeroOrden = "0F-M2-0001",` → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.

Línea 315: `Cliente = "Constructora del Norte",` → asigna el cliente de la orden utilizada en el escenario práctico del punto.

Línea 316: `FechaCreacion = DateTime.UtcNow,` → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.

Línea 317: `FechaEntrega = DateTime.Today.AddDays(14),` → establece una fecha de entrega prevista catorce días después de la fecha local actual.

Línea 318: `Estado = "Pendiente",` → inicializa la orden con el estado Pendiente, coherente con el flujo del laboratorio.

Línea 319: `Observaciones = "Orden de validación M2",` → añade texto opcional para comprobar el mapeo de Observaciones.

Línea 320: `Planchas =` → inicia el inicializador de la colección de planchas asociadas a la orden.

Línea 321: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 322: `new PlanchaAcero` → crea una nueva plancha dentro del inicializador de la colección Planchas de la orden.

Línea 323: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 324: `Espesor = 10.5,` → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.

Línea 325: `Ancho = 1500,` → asigna el ancho de la plancha usada en el escenario de persistencia.

Línea 326: `Largo = 3000,` → asigna el largo de la plancha usada en el escenario de persistencia.

Línea 327: `Peso = 371.250m,` → asigna un peso decimal con tres posiciones para validar la precisión configurada.

Línea 328: `Activa = true,` → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.

Línea 329: `}` → cierra el bloque de código actual.

Línea 330: `},` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 331: `Detalle = new DetalleOrden` → crea el detalle uno-a-uno que se persistirá junto con la orden principal.

Línea 332: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 333: `ComposicionQuimica = "C: 0.45%, Mn: 0.75%",` → asigna la composición química utilizada para verificar la entidad DetalleOrden.

Línea 334: `TemperaturaColada = 1550.5,` → asigna la temperatura de colada del detalle de fabricación.

Línea 335: `Notas = "Colada principal"` → asigna las notas opcionales del detalle asociado a la orden.

Línea 336: `},` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 337: `Certificado = new CertificadoCalidad` → crea el certificado uno-a-uno asociado a la orden del escenario.

Línea 338: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 339: `NumeroCertificado = "CERT-0001",` → asigna el identificador natural del certificado de calidad.

Línea 340: `FechaEmision = DateTime.Today,` → asigna la fecha de emisión del certificado usado en la prueba.

Línea 341: `OrganismoCertificador = "Laboratorio Aceria"` → asigna el organismo que emite el certificado de calidad.

Línea 342: `},` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 343: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 345: `var aleacion = new Aleacion` → crea la aleación que se enlazará con la orden mediante la entidad intermedia OrdenAleacion.

Línea 346: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 347: `Nombre = "AISI 1045",` → asigna el nombre comercial de la aleación utilizada en el escenario.

Línea 348: `Codigo = "A1045",` → asigna el código natural único de la aleación.

Línea 349: `PorcentajeCarbono = 0.45,` → asigna el porcentaje de carbono dentro del rango permitido por la restricción del modelo.

Línea 350: `PorcentajeManganeso = 0.75,` → asigna el porcentaje de manganeso dentro del rango permitido por la restricción del modelo.

Línea 351: `Descripcion = "Acero medio en carbono"` → asigna una descripción opcional a la aleación.

Línea 352: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 353: `orden.OrdenesAleaciones.Add(new OrdenAleacion` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 354: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 355: `Aleacion = aleacion,` → conecta la entidad intermedia `OrdenAleacion` con la aleación creada para el escenario.

Línea 356: `CantidadUtilizada = 1500.500m,` → registra la cantidad utilizada con tres decimales, coherente con `HasPrecision(18, 3)`.

Línea 357: `EstadoRelacion = "Activa"` → marca como activa la relación entre la orden y la aleación.

Línea 358: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 359: `context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 360: `context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 361: `context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el `DbContext`.

Línea 363: `var entity = context.Model.FindEntityType(typeof(OrdenFabricacion))!;` → obtiene los metadatos EF Core de `OrdenFabricacion` para inspeccionar claves, índices o filtros del modelo construido.

Línea 364: `global::System.Console.WriteLine($"2.7 OK | Fluent API | Estado default: {entity.FindProperty(nameof(OrdenFabricacion.Estado))?.GetDefaultValue()}");` → escribe en consola la evidencia usada para verificar el comportamiento del estado.

Línea 365: `}` → cierra el bloque de código actual.

Línea 366: `}` → cierra el bloque de código actual.